

Funciones autorizadas - Hilos POSIX

Esta versión incluye un diagrama explicativo (vectorial) y, para cada función, una tabla con qué se le envía, qué representa cada argumento y por qué a veces se pasa NULL.

Hilos (pthread_create / pthread_join)

pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void **retval) --
thread*: salida (id). attr: atributos defaults
start_routine: puntero a función. arg: argumento
pthread_join(pthread_t thread, void **retval) --

Mutex y Condiciones

pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr) --
attr NULL = atributos por defecto
pthread_mutex_lock(pthread_mutex_t *mutex) --
pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex) --
libera mutex mientras espera; al despertar v

IO y Tiempo

gettimeofday(struct timeval *tv, struct timezone *tz) --
tv: salida; tz: comúnmente NULL (no usado)
usleep(useconds_t usec) -- microsegundos
write(int fd, const void *buf, size_t count) --
fd: descriptor (1 = STDOUT). buf: datos. count: cantidad

Memoria y Cadenas

malloc(size_t size) -- bytes a reservar
free(void *ptr) -- liberar puntero
strcmp(const char *s1, const char *s2) / strlen(const char *s) --

Leyenda: NULL = usar valores por defecto o ignorar salida. Flechas y callouts muestran qué

Explicación por función: qué se le envía y por qué usar NULL

pthread_create

Firma:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void **retval)
```

Argumento	Qué es	Qué debe contener	Ejemplo / NULL
pthread_t *thread	ID del hilo creado (salida)	Puntero a variable pthread_t donde se guardará el id	&hilo
const pthread_attr_t *attr	Atributos del hilo (stack, detach, etc.)	Puntero a atributos o NULL para atributos por defecto	NULL (uso más común)
	Puntero a la función que ejecutará el hilo	Dirección de la función (debe devolver void *)	funcion_hilo

void *(*start_routine) (void *)			
void *arg	Argumento de usuario que recibe start_routine	Puntero a dato (struct/ int / NULL si no hay datos)	&id o NULL

Por qué usar NULL en attr: la mayoría de programas no necesita atributos personalizados; usar NULL evita configurar manualmente pthread_attr_t.

Ejemplo breve:

```
pthread_t hilo;
int id = 1;
pthread_create(&hilo, NULL, funcion_hilo, &id);
```

Salida terminal:

```
Hola desde el hilo 1
```

pthread_join

Firma:

```
int pthread_join(pthread_t thread, void **retval);
```

Argumento	Qué es	Qué debe contener	Ejemplo / NULL
pthread_t thread	ID del hilo que queremos esperar	Variable pthread_t con el id del hilo	hilo
void **retval	Dirección donde guardar el valor devuelto por start_routine	Puntero a void* que recibirá el retorno; o NULL para ignorarlo	NULL (si no necesitamos el resultado)

Ejemplo: si el hilo devuelve malloc(...) y queremos recuperarlo, pasamos la dirección de un puntero para recogerlo; si solo sincronizamos, pasar NULL es habitual.

```
Realizando tarea...
Resultado del hilo: 42
```

pthread_mutex_init / lock / unlock / destroy

Firma init:

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
```

Argumento	Qué es	Qué pasar	NULL?
<code>pthread_mutex_t *mutex</code>	Dirección del mutex a inicializar	<code>&mutex</code>	No
<code>const pthread_mutexattr_t *attr</code>	Atributos del mutex	Puntero a atributos o NULL	NULL = atributos por defecto

Bloqueo/desbloqueo: `pthread_mutex_lock(&mutex)` toma el mutex;
`pthread_mutex_unlock(&mutex)` lo libera.

Destruir: `pthread_mutex_destroy(&mutex)` libera recursos asociados.

Ejemplo salida esperada:

```
Contador final: 200000
```

`pthread_cond_*` (variables de condición)

Firmas clave:

```
pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *attr)
pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex)
pthread_cond_timedwait(pthread_cond_t *cond, pthread_mutex_t *mutex,
pthread_cond_broadcast(pthread_cond_t *cond)
```

Argumento	Qué es	Qué pasar	NULL?
<code>pthread_cond_t *cond</code>	Variable de condición	<code>&cond</code>	No
<code>pthread_mutex_t *mutex</code>	Mutex asociado que protege la condición	<code>&mutex</code>	No
<code>const struct timespec *abstime</code>	Tiempo absoluto de expiración	Puntero a <code>timespec</code> para timeout	Sí, si no usamos timeout (no pasar)

Importante: `pthread_cond_wait` libera el mutex mientras el hilo espera y lo vuelve a bloquear al despertarse — por eso debe llamarse siempre con el mutex tomado.

```
Producido: 1
Consumido: 1
Producido: 2
Consumido: 2
...
```

gettimeofday

Firma:

```
int gettimeofday(struct timeval *tv, struct timezone *tz);
```

Argumento	Qué es	Qué pasar	NULL?
struct timeval *tv	Estructura a rellenar con segundos y microsegundos	&tiempo	No
struct timezone *tz	Información de zona horaria (obsoleta)	Puntero a timezone o NULL	NULL (muy común)

Segundos: 1622551234 Microsegundos: 123456

usleep

Firma:

```
int usleep(useconds_t usec);
```

Se pasa el número de microsegundos (ej. 2000000 para 2 segundos).

Inicio Pasaron 2 segundos

write

Firma:

```
ssize_t write(int fd, const void *buf, size_t count);
```

Argumento	Qué es	Ejemplo típico
int fd	Descriptor de fichero (1 = STDOUT, 2 = STDERR)	STDOUT_FILENO
const void *buf	Puntero a los datos a escribir	mensaje
size_t count	Nº de bytes a escribir	strlen(mensaje)

Hola Mundo

malloc / free

Firmas:

```
void *malloc(size_t size);  
void free(void *ptr);
```

malloc recibe el número de bytes a reservar. free recibe el puntero devuelto por malloc (o NULL, que es seguro — free(NULL) es no-op).

0 2 4 6 8

printf / fprintf

Ambas reciben un `format` (cadena) y argumentos variables. `fprintf` añade además el stream objetivo (ej. `stderr`).

Mensaje a pantalla Mensaje a errores

strcmp / strlen / atoi / memset

Funciones de cadena y utilidades:

Función	Qué recibe	Notas
<code>strcmp(const char *s1, const char *s2)</code>	dos punteros a cadenas terminadas en '\0'	No pasar punteros NULL
<code>strlen(const char *s)</code>	puntero a cadena	retorna longitud (<code>size_t</code>)
<code>atoi(const char *nptr)</code>	cadena numérica	usar <code>strtol</code> para validación
<code>memset(void *s, int c, size_t n)</code>	buffer, valor byte, número de bytes	útil para inicializar buffers (ej. 0)

str1 y str2 son iguales str1 va antes que str3

Nota sobre imágenes: intenté generar una imagen externa para insertar en el PDF pero la generación automática de imágenes no funcionó por falta de créditos. En su lugar he incluido un diagrama vectorial (SVG) dentro del propio PDF para explicar visualmente los argumentos y el uso de `NULL`. Si prefieres una imagen externa más elaborada (PNG/SVG) puedo generarla cuando haya créditos disponibles o si tú subes una imagen de referencia.

¿Quieres que haga más cambios (más ejemplos sobre cómo pasar estructuras como argumento a los hilos, diagramas de secuencia entre hilos, o una versión DOCX/PPTX)?